

Aluno: João Paulo Oliveira Santana

Prontuário: SP3295893

Exercícios 1. Conceito de modularização

Explique com suas próprias palavras o que significa **modularização** em programação. Qual é a principal vantagem de dividir um programa grande em módulos menores?

Modularização é dividir partes do sistema em partes menores chamadas de módulos, onde cada um é responsável por um papel específico do sistema, dessa forma ajudando bastante na organização do código, divisão de tarefas entre integrantes de um grupo, manutenção do software e escalabilidade do sistema.

2. Coesão

Defina o conceito de **coesão**. Explique a diferença entre **alta coesão** e **baixa coesão** e indique qual delas é desejável em um projeto de software e por quê.

Coesão é o grau com que os elementos dentro de um módulo estão relacionados e focados em realizar uma única tarefa ou objetivo; um trecho de código que faz só uma coisa e todos os comandos trabalham para o mesmo propósito é conhecido como de Alta Coesão; já se o módulo possui funções que atuam em diferentes responsabilidades e sem relação direta é conhecida como Baixa Coesão.

Em um projeto de software é mais aconselhável ter alta coesão por facilitar na organização do sistema.

3. Análise de coesão

Considere uma classe chamada Usuario que possui os seguintes métodos:

```
class Usuario:
```

```
    def cadastrar_usuario(self):
```

```
        pass
```

```
        def        alterar_usuario(self):
```

```
            pass
```

```
        def        calcular_frete(self):
```

```
            pass
```

```
def        gerar_relatorio_vendas(self):  
pass
```

```
def enviar_email_usuario(self):  
    pass
```

Analise a classe e responda:

a) Ela apresenta alta ou baixa coesão?

Ele possui baixa coesão por ter métodos que não tem correlação direta com os usuários.

b) Quais métodos parecem não pertencer à responsabilidade principal da classe?
São os métodos: `calcular_frete(self)` e `gerar_relatorio_vendas(self)`.

c) Como você reorganizaria esse código para melhorar sua coesão?

Retiraria os métodos que não se correlacionam com o usuário e o código ficaria assim:

```
class Usuario:  
    def cadastrar_usuario(self):  
        pass  
  
    def        alterar_usuario(self):  
pass  
  
    def enviar_email_usuario(self):  
        pass
```

4. Responsabilidade Única – SRP

Explique a relação entre **alta coesão** e o **Princípio da Responsabilidade Única (Single Responsibility Principle – SRP)**. Uma classe pode possuir vários métodos e ainda apresentar alta coesão? Justifique e dê um exemplo. O material utiliza a classe **Estoque** para mostrar que vários métodos podem coexistir quando todos trabalham com a mesma responsabilidade.

Um código com alta coesão apresenta o SRP pois todos os métodos dentro da classe atuam em um único objetivo dentro do sistema, logo uma classe que tem métodos para diferentes objetivos não é coeso e não segue o SRP.

```
Class Estoque(self):
```

```
    def adicionar_item(self):
```

```
        pass
```

```
    def retirar_item(self):
```

```
        pass
```

```
    def consultar_item(self):
```

```
        pass
```

5. Conceito de acoplamento

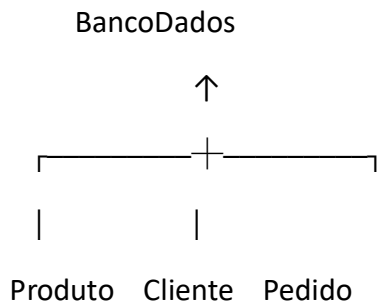
Defina **acoplamento** em programação. Explique a diferença entre **alto acoplamento** e **baixo acoplamento** e indique qual situação normalmente é mais adequada para facilitar a manutenção de um sistema.

Acoplamento é o grau de dependência entre dois ou mais componentes de um sistema, como classes, funções ou módulos. Um código com alto acoplamento são extremamente dependentes e uma alteração pode quebrar todo o sistema; já um código com baixo acoplamento os módulos são independentes, podendo fazer alterações, testes ou substituições sem afetar o restante do sistema.

Para a manutenção é mais aconselhável ter um sistema com baixo acoplamento.

6. Problemas do alto acoplamento

Considere um sistema composto pelas classes:



Suponha que Produto, Cliente e Pedido conheçam diretamente detalhes internos da classe BancoDados.

Explique pelo menos **quatro problemas** que essa estrutura pode causar caso a implementação de BancoDados precise ser modificada. O material relaciona o alto acoplamento a problemas como dificuldade de manutenção, testes, reutilização e substituição de componentes.

O alto acoplamento pode causar:

- ⑩ Dificuldade de manutenção: cada problema que der no sistema para arrumar pode alterar completamente como o código funciona e resultar em quebras do sistema.
- ⑩ Maior impacto das alterações: como todas as classes estão conectadas, cada alteração pode mudar drasticamente o sistema e possivelmente quebrar o código.
- ⑩ Testes mais difíceis: como todas as classes estão conectadas, se ocorrer um problema no sistema dificilmente será encontrado em qual parte específica dos módulos é responsável por ele.
- ⑩ Código mais rígido: é mais difícil executar mudanças em um código onde seus elementos estão fortemente interligados.

7. Identificação de acoplamento

Observe: banco = []

class Produto:

```

def cadastrar(self, nome):
    banco.append(nome)
  
```

```
def listar(self):  
    return banco
```

Responda:

a) A classe Produto possui dependência de algum elemento externo?

Ela possui uma interligação com o elemento externo banco.

b) O uso direto da variável global banco aumenta ou reduz o acoplamento?

O uso da variável global aumenta o acoplamento do sistema.

c) Que problema poderia ocorrer se o sistema deixasse de utilizar uma lista e passasse a utilizar um banco de dados?

A classe retornaria um valor que não existiria mais, o que possivelmente quebraria o sistema caso fosse usada.

d) Proponha uma forma de melhorar essa implementação.

A função poderia retornar os valores em forma de lista sem atribuir a uma variável.

8. Injeção de dependência e baixo acoplamento Analise:

class Produto:

```
def __init__(self, repositorio):  
    self.repositorio = repositorio
```

```
def cadastrar(self, nome):  
    self.repositorio.salvar(nome)
```

class RepositorioProduto:

```
def salvar(self, nome):  
    print(f"Salvando {nome}")
```

```
repositorio = RepositorioProduto() produto =
```

```
Produto(repositorio)
```

```
produto.cadastrar("Notebook")
```

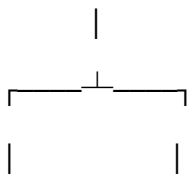
Explique por que receber o repositório externamente pode contribuir para reduzir o acoplamento da classe Produto. Quais vantagens essa estratégia oferece para **manutenção, substituição de componentes e testes**? Esse modelo é apresentado no material como uma forma de fornecer a dependência externamente em vez de fazer a classe controlar todos os detalhes de sua criação.

Como a classe Produto recebe o repositório de forma externa, ele não precisa se preocupar em controlar todos os detalhes de sua criação, o que reduz o seu acoplamento com outras classes e módulos. Dessa maneira é mais fácil realizar mudanças e testes, já que agora eles serão menos prováveis de quebrar o sistema.

9. Coesão × Acoplamento

Explique a diferença fundamental entre **coesão** e **acoplamento**, completando conceitualmente:

MODULARIZAÇÃO



COESÃO ACOPLAMENTO



Dentro do Entre Módulos

Módulo



Alta

Baixo

Objetivo de Projeto

Indique:

a) Qual conceito está relacionado ao que acontece **dentro de um módulo**?

Coesão.

b) Qual está relacionado à dependência **entre módulos**?

Acoplamento.

c) Devemos buscar coesão alta ou baixa?

Devemos buscar alta coesão.

d) Devemos buscar acoplamento alto ou baixo?

Devemos buscar baixo acoplamento.

10. Exercício prático – Refatoração

Considere: class Sistema:

```
class Estoque(self):  
    def cadastrar_produto(self, nome):    print("Cadastrando  
produto:", nome)
```

```
    def        excluir_produto(self,        nome):  
print("Excluindo produto:", nome)
```

```
class cliente(self):  
    def cadastrar_cliente(self, nome):  
        print("Cadastrando cliente:", nome)
```

```
class pagamento(self):  
    def calcular_pagamento(self, valor):    print("Calculando  
pagamento:", valor)
```

```
class relatorio(self):
```

```
def gerar_relatorio(self):  
print("Gerando relatório")
```

```
class email(self):  
    def enviar_email(self):  
print("Enviando e-mail")
```

O programa funciona, mas apresenta problemas de organização.

Faça uma **refatoração**, dividindo as responsabilidades em classes ou módulos mais apropriados. Por exemplo:

```
sistema/  
|  
├─ produto.py  
├─ cliente.py  
├─ pagamento.py  
├─ relatorio.py  
├─ email.py  
└─ main.py
```

Depois responda:

a) Por que a classe Sistema original apresenta **baixa coesão**?

Porque dentro do módulo main tem todos os métodos de diferentes partes do sistema.

b) Como a divisão em módulos melhora a coesão?

Com a divisão de cada método em classes específicas, a coesão do programa aumenta muito já que respeita o conceito de SRP.

c) Como você organizaria as dependências para evitar alto acoplamento?

Tentaria fazer com que cada método fosse o mais independente possível.

d) Escreva uma possível implementação em **Python** da solução proposta.

```
class Estoque(self):  
    def cadastrar_produto(self, nome):    print("Cadastrando  
produto:", nome)
```

```
    def        excluir_produto(self,        nome):  
print("Excluindo produto:", nome)
```

```
class cliente(self):  
    def cadastrar_cliente(self, nome):  
        print("Cadastrando cliente:", nome)
```

```
class pagamento(self):  
    def calcular_pagamento(self, valor):    print("Calculando  
pagamento:", valor)
```

```
class relatorio(self):  
    def        gerar_relatorio(self):  
print("Gerando relatório")
```

```
class email(self):  
    def        enviar_email(self):  
print("Enviando e-mail")
```

O objetivo não é eliminar completamente as dependências entre módulos: o próprio material ressalta que sistemas reais precisam de relacionamentos. O objetivo é **reduzir dependências desnecessárias e controlar as necessárias**.

